```
countUsingFile = (file, strategy) ->
  readFile file, strategy
  fs.watch file, (-> readFile file, strategy)

init = ->
  countUsingFile 'partygoers.txt', attendeesCounter
  countUsingFile 'friends.txt', friendsCounter

  server = http.createServer (request, response) ->
    switch request.url
      when '/'
        response.writeHead 200, 'Content-Type': 'text/html'
        response.end view
      when '/count'
        response.writeHead 200, 'Content-Type': 'text/plain'
        response.end "#{attendees + friends}"
  server.listen 8080, '127.0.0.1'
  console.log 'Now running at http://127.0.0.1:8080'

  clientScript = coffee.compile '''
  get = (path, callback) ->
    req = new XMLHttpRequest()
    req.onload = (e) -> callback req.responseText
    req.open 'get', path
    req.send()

  showAttendees = ->
    out = document.querySelector '#how-many-attendees'
    get '/count', (response) ->
      out.innerHTML = "#{response} attendees!"

  showAttendees()
  setInterval showAttendees, 1000
  '''
  view = """
  <!doctype html>
  <title>How many people are coming?</title>
  <body>
  <div id='how-many-attendees'></div>
  <script>
  #{clientScript}
  </script>
  </body>
  </html>
  """

init()
```

Set up the two source files. Sample files are provided in the downloadable source.

Listen handler on port 8080.

Client/browser CoffeeScript.

HTML document as heredoc, rendered by HTTP handler.

This entire program is more complicated than anything you've seen before in CoffeeScript. But everything in it is either a general syntactic feature of CoffeeScript or a technique for using functions that you've learned in this chapter. It's apparent, though, that if this program were to get larger, then